

dbt is set to NULL to indicate that this is the most recent snapshot row. With invalidateharddeletes enabled, records that are deleted from the source will have their last dbtvalidto updated to reflect the timestamp when they were removed instead of NULL.

- For the previous version of the record, dbtvalidto is updated to match the dbtvalidfrom timestamp of the new record, marking the end of the previous record's validity.

dbt Snapshots, by default, are loaded incrementally. Records that have changed are picked up each time the snapshot operation runs.

Snapshot Model Type Features

Snapshot models are categorized based on the features and techniques used when working with snapshot data. Here are the main types of snapshot models:

- dbt Snapshot: The foundational snapshot model created in RAW over a single table. This model captures the state of the source data over time but is not intended for direct analysis.
- Over snapshot: A model built directly on top of the dbt Snapshot model, adding additional logic or transformations for downstream analysis.
- Spined dates: A model that integrates one or more snapshot models with a date spine, allowing for time-based analysis or alignment with other datasets.
- History rebuild: A model that uses snapshot data to recreate historical states with the same logic as an SCD (Slowly Changing Dimension). These models allow for rebuilding history if columns or business logic change.

Snapshot Model Type Examples:

- Here are examples of snapshot models with the variation of features that help determine the type:

dbt Snapshot	Over Snapshot	Spined Dates	History ReBuild	Example
X				dimsubscriptionsnapshot
X				dimusersnapshot
	X			dimnamespacehist
	X			dimuserhist
	X	X		dimsubscriptionsnapshotmodel
	X	X	X	dimsubscriptionsnapshotbottomup
	X	X	X	dimusersnapshotbottomup

Testing Snapshots

Testing can be performed locally by setting SNOWFLAKESNAPSHOTDATABASE to the name of your personal <USERROLE>PREP database in your zshrc file during the testing period. This

configuration allows you to test new snapshots, and when combined with cloning, enables testing changes to existing snapshots.

Snapshots and GDPR

Sometimes the data team receives requests to delete personal data from the Snowflake Data Warehouse, because of GDPR. To address these deletions, we use dbt macros. A macro scans all applicable data that needs to be removed, this also applies to snapshot tables. These macros could be used manually following this issue template and are also scheduled to be run in Airflow following this runbook and handbook page.

There are 2 flavours:

1. The GDPR deletion request applies to all GitLab sources, and therefore all tables in the data warehouse need to be checked and updated. Macro. Use this macro if the issue that requesting a GDPR deletion states Deletion Request (Full) (in either the title or issue description).
2. The GDPR deletion request applies to only GitLab.com related sources and therefore only GitLab.com related tables in the data warehouse need to be checked and updated. Macro. Use this macro only if the issue that requesting a GDPR deletion states Deletion Request (GitLab.com Only) (in either the title or issue description).

Specific to the second flavour, check when creating a new snapshot model or rename an existing snapshot model, if the dbt macro covers the models involved. Check if the filtering in the macro applies to the applicable snapshot tables in case of a GDPR deletions request for GitLab.com only related sources.

Make snapshots table available in prod database

We create source models based on snapshots, and we recommend using ref instead of source for referencing them. Using ref ensures that dbt manages dependencies effectively, allowing the DAG to track relationships between models. This ensures the correct build order during dbt runs. By contrast, source is more static and bypasses dbt's dependency tracking.

Building models on top of snapshots

Models built on top of snapshots are designed for historical analysis. These models incorporate snapshot data and can include logic to join multiple snapshots or derive new metrics. By combining snapshots and applying business rules, they provide a comprehensive historical view of data. A key feature of these models is date spining, which allows for easy analysis of record values for any day in history by generating rows for all dates between dbtvalidfrom and dbtvalidto. These models are often configured as incremental since the underlying dbt snapshots are append-only and not modified. If the structure or logic of a published snapshot model changes, a full refresh may be required.

In date spining, a snapshot model is joined with a date table using the dbtvalidfrom and dbtvalidto fields. This process generates a table with one record for each day, representing the state of records active on a specific day (referred to as the snapshotdate).

If you are using date spining to generate record for each day, consider materializing the model

as incremental.

Testing Downstream Impact

To manually review the downstream impacts a change to a model may have use the asset and field level lineage in MonteCarlo.

Dropping dbt Models

To drop dbt models, remove the relevant files in your local IDE, commit the changes, and push them as part of a merge request to run in the CI pipelines.

Note that Snowflake tables in Production are not automatically removed and must be handled separately by the Data Platform team.

The MR author should create a follow-up issue and assign it to the Data Platform team to complete the table removal.

In some cases, tables may need to be retained for historical purposes even after the dbt models are removed, which means they will no longer be updated but remain in place for reference.

Here is an Example MR that shows Models being deprecated with some of the tables being retained in the database.

Model Efficiency

A model's efficiency is a measure of how well the model uses the Snowflake resources to produce the model. At this time, it is not a measure of the queries used as we have not found a way to procedurally and reliably quantify the actions taken within a query. The efficiency score of a model can be determined for each model invocation and is based on three numbers determined from the queries executed by the model. The component numbers are intended to provide an insight into where to investigate when the overall number does not meet the intended targets. These scores can be aggregated to show the overall efficiency of a grouping of models, such as a run or day.

Method

For each model the queries executed are first filtered and aggregated. Only the specific query types of CREATEVIEW, INSERT, DELETE, CREATETABLEASSELECT, MERGE, CREATEVIEW, SELECT, EXTERNALTABLEREFRESH are considered and the query properties of bytesscanned, bytesspilledtoremotestorage, bytesspilledtolocalstorage, partitionstotal, partitionsscanned are aggregated for calculation. Once aggregated the following metrics are calculated:

Local Storage Efficiency

$$LE = \min\left\{\frac{s-S}{s}, 0\right\}$$

- Where (S) is the model Bytes Spilled to Local Storage
- Where (LE) is the model Local Storage Efficiency
- Where (s) is the model Bytes Scanned

The metric is calculated as the model bytes scanned less the model bytes spilled to local storage divided by the model bytes scanned and limited to values between 0 and 1. This calculation allows for a number that is independent of other models but still comparable to other models.

Remote Storage Efficiency

$$E_r = \min\left\{\frac{s - s_r}{s}, 0\right\}$$

- Where (s_r) is the model Bytes Spilled to Remote Storage
- Where (E_r) is the model Remote Storage Efficiency
- Where (s) is the model Bytes Scanned

The metric is calculated as the model bytes scanned less the model bytes spilled to remote storage divided by the model bytes scanned and limited to values between 0 and 1. This calculation allows for a number that is independent of other models but still comparable to other models.

Partition Scan Efficiency

$$E_p = \text{if } p > 1 \text{ then } \min\left\{\frac{p - s_p}{p}, 0\right\} \text{ else } 1$$

- Where (s_p) is the model Partitions Scanned
- Where (E_p) is the model Partition Scan Efficiency
- Where (p) is the model Total Partitions

If there is more than one model partition then the metric is calculated as the model total partitions less the model partitions scanned divided by the model total partitions and limited to values between zero and one, otherwise the metric value is set to one. This calculation allows for a number that is independent of other models but still comparable to other models. It is expected that most models will not be able to achieve a partitions scan efficiency value of one as some number of partitions will always need to be scanned, but efforts should be made to improve the metric as much as possible.

Efficiency Score

$$E = [(E_l \cdot w_l) + (E_r \cdot w_r) + (E_p \cdot w_p)]100$$

- Where (E) is the model Efficiency Score
- Where (E_p) is the model Partition Scan Efficiency
- Where (E_r) is the model Remote Storage Efficiency
- Where (E_l) is the model Local Storage Efficiency
- Where (w_p) is the model Partition Scan Efficiency weight
- Where (w_r) is the model Remote Storage Efficiency weight
- Where (w_l) is the model Local Storage Efficiency weight

The compound score is calculated as the weighted average of the Local Storage Efficiency, Remote Storage Efficiency, and Partition Scan Efficiency metrics. The metric weights are determined arbitrarily by the needs and focus of the business.

Usage

Model efficiency can be looked at for an individual model or for a collection of models. When looking at a collection of models, it is recommended that efficiency metrics and scores be weighted, for example by the total bytes scanned, across the models in the collection. When the metrics and score are below target values, each metric can indicate an area of exploration to improve the efficiency: Local Storage Efficiency and Remote Storage Efficiency indicate insufficient memory for the bytes processed by the model and low Partition Scan Efficiency indicates that the model may not be set up to prune partitions as part of the needed table scans.

Reporting

To report on the overall efficiency of dbt models for the organization the latest invocation of each model is used and weighted by the total bytes scanned. This method provides a view over time of the changes to the aggregate efficiency of all of the models and represents a lagging indicator of the effect of changes to models and new models added to the collection. Each metric and score are available in the reporting so that the cause of changes can be drilled into and areas of improvement can be identified.

Model Performance

Performance should be balanced between model execution times, which directly affect Snowflake Credit spend, and developer time, which impacts how long a developer is waiting during development, troubleshooting, and bug fixes. Generally, model execution should strike a balance between running as fast as possible and with as little cost as possible. This can be a difficult balance as some of the methods to improve run time also increase the cost, however there are methods and techniques that can be used to achieve both improved run time and decreased cost. As the number and complexity of models grow, general model run time performance criteria is needed. The overall performance goals for the data warehouse are that:

- All models should individually execute in less than 60 minutes.
- All models should be run on the smallest warehouse allowable

Categorization

To provide a basis for communication and criteria for applying methods of improvements, categorizations for model and table size have been developed as follows:

Model Run Time

Defined as the total run time of all queries associated with the execution of a model.

Categorization Details

Categorization Execution Seconds > Execution Seconds >=			
XS	0	60	
S	60	300	
M	300	900	
L	900	1800	

XL	1800	3600	
XL+	3600		

Model Size

Defined as the rows and bytes of the table produced by the model. Categorization Details

Categorization	Rows >	Rows <=	GB >	GB <=	
-----	-----	-----	-----	-----	
XS	0	1,000,000	0	1	
S	1,000,000	10,000,000	1	10	
M	10,000,000	100,000,000	10	100	
L	100,000,000	1,000,000,000	100	1,000	
XL	1,000,000,000		1,000		

Model Efficiency

Defined as the total bytes spilled compared to the total bytes written of all the queries of a model. Categorization Details

Categorization	GB Spilled Local	GB Spilled Remote	
-----	-----	-----	
Good	0	0	
Acceptable	<= 5 GB Written	0	
Poor	> 5 GB Written	0	
Very Poor		> 0	

Methods

When looking for opportunities to improve model performance, the overall impact of the effort should be kept in mind. For example, spending a lot of effort on a model with a run time classification of S and model efficiency classification of Poor is not likely to yield a significant improvement on overall dbt invocations. The methods provided in this section are meant to be a guide for performance improvements, they are not a strict set of rules that can be enforced without thought to their implication.

Model Materialization

The way a model is materialized will directly impact how long the model runs during execution. Some materializations will improve a model's performance, but decrease the performance of downstream models and queries while others will improve performance at the cost of increased complexity. When possible, quantifiable criteria are used to guide the materialization recommendations, but it is more likely that the developer will have to use their best judgement to select the materialization.

- View models should be used when the transformations are light or the impact of the transformations are minor on the performance of downstream models.
- Ephemeral models should be used when a transformation is simple and needs to be used across multiple other models.
- Table models should be used when there is complex query logic that negatively impacts

downstream queries if it were to be performed with each query.

- Incremental models should be used when the table is a size M or larger, the model run time is a size L or larger, and existing source data changes very little day to day.

- Incremental models should be set to never full refresh if the table is a XL size or larger, the source data does not update old records, and there is low risk of the schema of the source data changing.

- Incremental models can use a temporary table or a view for the incremental load and may have different performance. Each option should be evaluated when creating a new incremental model. Further details can be found [here](#).

Query Optimization

Query optimization involves looking at the way the query executes and making changes that will allow the model to run more efficiently. General guidance for optimizing query performance is as follows:

- Reduce spilling to local and remote storage by reducing the columns and rows that are used in the query
- Ensure that the order JOIN clauses are operating on as few rows as feasible
- Minimize fan out of rows and cartesian results in JOIN clauses

Test Optimization

By default uniqueness and notnull should be tested on all primary keys, however tests should be able to complete in a reasonable time. If a long-running test is detected there are some strategies that can be used to remove the test or improve performance.

- Is the test checking for uniqueness on a model which already has a QUALIFY statement on that field? In this case uniqueness is built into the model and does not need to be tested although care should be taken that further changes do not alter this.

- Is the test checking for uniqueness on an incremental model which already has that field defined as the uniquekey? Any attempt to insert a duplicate key will result in a processing error so testing may not be required.

- If the table is very large then the lookback period can be specified in the test config, for example:

yaml

datatests:

- unique:

- config:

- where: "createdat >= DATEADD('day',-3,CURRENTDATE())"

Clustering

The application of clustering, and automatic reclustering, will be very dependent on the situation and would typically be placed on the source tables in the lineage of the model where a performance increase is desired. Clustering should be considered in the following circumstances:

- Partition pruning is poor in the model build query.
- There are specific and consistent columns used for filtering and joining in queries against the source table.

Adjusting the Warehouse size

Adjusting the warehouse size for improved performance is not straight forward as there are many factors that go into how a query will perform. Warehouse size adjustment may be considered under the following circumstances:

- Increasing Warehouse Size
 - The model efficiency is regularly categorized as poor or very poor.
- Decreasing Warehouse Size
 - The model efficiency is good and the model run time is XS.

Other potential methods

- Search Optimization
 - <https://community.snowflake.com/s/article/Search-Optimization-When-How-To-Use>

System Level Optimization

There are a number of other potential system level optimizations that can be considered to improve overall performance.

- Invocation Threads
- Max Concurrent Query level
- Auto Scaling Policy
- Warehouse Max Clusters

Guidance for Checking Model Performance

Retrieve Model Performance Statistics

Using the modelbuildperformance runbook retrieve the model performance statistics.

Check model Run Time

If the model run time is less than L then the effort needed to improve the model is not likely to yield significant returns at this time and the model can be left as it is.

Check the Model Efficiency

A Model Efficiency worse than Acceptable, represents significant spillage thus reducing the performance because of the Remote and Local disk reads.

- Reduce the amount of data needed to be stored in memory during the query execution.
 - Reduce number of Columns used
 - Reduce number of Rows used
 - Check Filter order

- Check Join order
- Check for Redundant Joins

Check for incremental viability

Not all models need to be incremental due to the increased design complexity.

- If the model loads new data regularly, but most of the data does not change it may be a good candidate for an incremental model.
- If the source data can be filtered in a way to return new or changed records, such as an updateddate field, it may be a good candidate for an incremental model.
- If the model represents the current state of data that changes it is not a good candidate for an incremental model.
- If the model is based on a snapshot model, it is not a good candidate for an incremental model.

Check never full refresh viability

- Event stream data that does not change is a prime candidate for never full refreshing.
- Check data source guidelines from the handbook.

Check Warehouse Size viability

Increasing the warehouse size will not always mean an increase in performance and an improvement in cost.

- If refactoring has not improved the Model Efficiency, an increase to the Warehouse Size may be warranted.
- If Model Efficiency is Good but the model is reaching the timeout limit for the warehouse then an increase to the Warehouse Size may be warranted.

The Snowflake warehouse sizing handbook page has guidelines on properly sizing dbt models.

Upgrading dbt

See the runbook for instructions on how to independently and asynchronously upgrade dbt.

Staying up to date

Our policy is that we should always be on a version of dbt-core that does have critical support. Check the linked schedule to see planned releases and support windows. For minor releases that are released while we are still on a version with critical support, we will evaluate on a quarterly basis to determine whether the minor release warrants an update.

When a major release happens, we should upgrade to the new major version before the second minor release on the new major version. So for example, we should be on v2.0.0 before v.2.2.0 is released. The extra time allowance is in place to account for breaking changes between major versions.

Scheduling a dbt upgrade

dbt upgrades should take place on a Tuesday of a week in which there are no major worldwide holidays or Family and Friends days. This is to enable enough time for team members to correct any breaking changes that weren't caught in testing without having to work through the weekend. In a worst case scenario, the upgrade can be rolled back on Wednesday so that normal operations can resume for the remainder of the week.

Specifying warehouse size on dbt model level

New product and non-product models will use a default 'L' warehouse size going forward.

Context: In order to decrease the runtime of the production dbt DAG, the product and non-product models have been consolidated into one Airflow task, MR. This means that the new default warehouse size will be 'L', and that if an 'XL' needs to be used, it needs to be specified on the dbt model level.

To specify the warehouse on the dbt model level (thus overriding the default warehouse), you need to add a config block to the model:

```
sql
{{ config(
    snowflakewarehouse=generatewarehouse('XL')
) }}
```

dimnote.sql is an example model you can refer to.

For more info on choosing the correct warehouse, please see the 'Check Warehouse Size viability' section of this page.

SECTION: enterprise-data/platform/gdpr-deletions.md

Supporting the GDPR process

Technical background on the process listed here: Runbook

A brief overview of the process:

```
mermaid
graph LR
    A[Driveloadgdpr-delete-requests] --> B[dbtgdprdeleterequets] --> C[Tableau Report] --> D[Logoutput.csv]
```

Driveloadgdpr-delete-requests

Standard Driveload process, looking for files in this folder.

dbtgdprdeleterequets

Modified DBT DAG. There are two key things to note here:

1. Run operation macro: `gdprbulkdelete`

- This macro reads in all the data provided in the Driveload process, and loops through one record at a time, running the `gdprdelete` or `gdprdeletetigitlabdotcom` macros (depending on what was provided in the Driveload file).

- Additionally, after processing each row, this process DELETES the source record from the `RAW.Driveload` table. As this process is designed to remove all the data for an e-mail address, we cannot store the source records long term.

2. DAG params: `--log-path gdprrunlogs --log-format json`

- This provides a better format for the log outputs so that we don't have to manipulate strings to extract this data.

- We also created a separate parsing path in `orchestration/uploaddbtfiletosnowflake.py` to parse and upload this json data separately to the other type of DBT logs.

- Finally, the `--path` parameter was created so that these logs are not mixed in with any of the general logs and so that we have a consistent format containing only the logs pertaining to the most recent run (this path is not persisted between runs)

Tableau report

Workbook, detailed here:

- Only thing to really note here is that the Data Source has a Custom SQL Query attached to it to try and filter out most of the unnecessary records from the batch.

- The filter excludes a few fields and only returns runs which have more than 100 logged records. The reasoning here is that when the process runs without any Driveload input it creates 50-70 rows, when there was an input it is 250+, so this gives a clean exclusion on records we don't need.

- In the runbook, it is detailed that we use the Tableau downloader to extract the data, however, if this proves problematic, and you have Snowflake access the data can also be extracted very easily through Snowflake.

SECTION: enterprise-data/platform/permifrost.md

Quick Links

Permifrost Project

PyPI

Permifrost

Permifrost is a Python tool for managing permissions on a Snowflake data warehouse. The main documentation for using the tool is available in the project and on PyPI.

Caveats

:whitecheckmark: What Permifrost is and what it is doing

Supports Snowflake only

Permifrost takes care of roles and privileges only

Objects that exist but are not in the config file do not lead to errors

Is a fully open-source package and everyone can contribute

:x: What Permifrost is not and what it is not doing

Object creation and deletion is not managed by permifrost

That is, if entire roles are removed from the roles.yml file, it will not delete them from Snowflake

Contributing

Development

Follow these steps to create and prepare your virtual environment.

bash

create a virtualenv

python -m venv ~/.venv/permifrost

activate your virtualenv

source ~/.venv/permifrost/bin/activate

install dependencies

pip install -r requirements.txt

pip3 install all the development dependencies

pip install -e '[dev]'

Once you've committed your changes, submit a merge request and update the default template.

Communication

For any additional question, contribution or ide, feel free to open a new issue in the Permifrost project and, if needed, tag @gitlab-data/permifrostmaintainers (the project maintainers)

If you prefer to announce or ask, Slack channel tools-permifrost is also a good option.

Release Process

The release process is described in the template Release process for Permifrost with using Release Permifrost template. For real life example, use Release 0.15.1 as a template.

We set a goal to release a new version of Permifrost at least twice per year. Meanwhile, check our release history.

Versioning

Permifrost uses Semantic Versioning 2.0.0 as its version number scheme.

Workflow

Permifrost uses tags to create its artifacts. Pushing a new tag to the repository will publish it as docker images and a PyPI package. For details, refer to the Release guideline.

Below is the entire workflow for Permifrost contribution and development:

mermaid
graph TD

```
ISSUE--Tag permifrost maintainers--> VAL
VAL--permifrost::1 Accepting Merge Requests-->MR
MR--permifrost::2 In-Progress--> MRD
MRD--permifrost::3 In-Review-->APR
APR--Yes-->APRYES[Adding to chanelog]
APR--Work needed-->MR
END((End))
subgraph Community
    RFC((Request for change))
    RFC--Create an issue-->ISSUE[Issue]
    MR[Merge request]
    MRD[Work is done]
end
subgraph Permifost maintainer
    VAL[Validate issue]
    APR{Code is approved?}
    APRYES--Release-->PyPi[PyPi]
    PyPi--share info about new release-->END
end
```

SECTION: enterprise-data/platform/python-guide.md

Python Guide

Motivation

It is the Data team's collective responsibility to promote, respect, and improve our Python Style Guide since not all the things can be caught up by the tools we are using. This deliberate attention makes a difference and does a lot to ensure high-quality code.

The main motivation to have comprehensive and useful Python guidelines is to ensure a high standard of code quality is maintained in our work. As we use this guideline as a support in that daily work, which is always changing, this guide is likewise always subject to iteration. In the long run this guide will help us to have a world-class code quality we should be proud of. All the changes are driven by our values.

Values

Campsite rule: As these guidelines are themselves in a constantly Work in Progress (WIP) status - if you work with any code style, hint, or guideline which does not currently adhere to the style guide - please submit a merge request with the relevant changes and tag the Data Platform team to update the guide.

Technology Standardisation

Starting in Jan 2022 all new custom python extracts should adhere to the Singer standard.

High-level guidelines

This section brings details on a high level of how to follow the best practices regarding Python code. Following recommendations, will ensure we are able to fully understand the advantage of high code quality and leverage our code base.

Zen of Python

It is difficult to resist writing a python guide without mentioning the Zen of Python, a cornerstone for ensuring the code quality on a high level.
It is a helpful mental exercise when you want to write outstanding Python without overlooking basic ideas.

bash

```
..$ python3
```

```
Python 3.8.6 (v3.8.6:db455296be, Sep 23 2020, 13:31:39)
```

```
[Clang 6.0 (clang-600.0.57)] on darwin
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> import this
```

```
The Zen of Python, by Tim Peters
```

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than right now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

PEP 8

PEP stands for Python Enhancement Proposal, and there are several of them. A PEP is a document that describes new features proposed for Python and documents aspects of Python, like design and style, for the community.

As per definition:

- > PEP 8, sometimes spelled PEP8 or PEP-8, is a document that provides guidelines and best practices on how to write Python code.
- > It was written in 2001 by Guido van Rossum, Barry Warsaw, and Nick Coghlan.
- > The primary focus of PEP 8 is to improve the readability and consistency of Python code.

Why We Need PEP 8?

- > Readability counts.
- >
- > · The Zen of Python
- >
- > Code is much more often read than it is written.
- >
- > · Guido van Rossum

Among many other things, it is vital to underscore the need to have a clean, clear and consistent code in order to be able to maintain and scale the codebase.

GitLab Zen of Python

As a supplement of the Zen of Python, we had an intention to underscore a couple of more viewpoints to ensure and keep our code base in a good shape. This is half a joke, half a truth, but provides good general overview for the high standards we want to leverage.

Here is our GitLab Zen of Python proposal:

- Gratitude and respect for PEP 8
- Insist on writing well-structured code
- Trust Pythonic way of thinking and coding, and introduce a good habit of using it on a daily basis
- Leverage and promote proper comments and documentation
- Always have Zen of Python on the top of your mind
- Boost usage of a modular code style over script-like approach

Probably a couple more of them are count:

- Advocate for proper naming of variables, classes, functions and modules
- Favor a modular code style over script-like approach
- Prefer using a virtual environment over existing interpreter

Specific guidelines

With a good framework definition on the high level of using the proper architectural approach

for designing and structuring the Python code, now it is time to do a deep dive and leverage details should make the difference between good and the outstanding code.

Project setup - Poetry

For project setup, we are using poetry.

Poetry is a python dependency management tool to manage dependencies, packages, and libraries in your python project. It will help you to make your project more simple by resolving dependency complexities in your project and managing to install and update for you.

Here is the set of command we are using to initiate and install Python (in our case it is 3.10.3), refer to the file onboardingscript.zsh.

One good example of how we define pyproject.toml file to run and execute the poetry setting is exposed · below:

```
bash
[tool.poetry]
name = "app"
version = "0.1.0"
description = "Automated Service Ping Metrics Check"
authors = ["{}"]
readme = "README.md"
homepage = "https://gitlab.com/gitlab-data/service-ping-metrics-check/README.md"
repository = "https://gitlab.com/gitlab-data/service-ping-metrics-check"

[tool.poetry.dependencies]
python = "^3.10"
fastapi = "^0.85.0"
uvicorn = "^0.18.3"
black = "^22.8.0"
pylint = "^2.15.3"
flake8 = "^5.0.4"
mypy = "^0.971"
vulture = "^2.6"
pytest = "^7.1.3"
python-decouple = "^3.6"
flatten-dict = "^0.4.2"
sqlparse = "^0.4.3"
snowflake-connector-python = "^2.7.12"

[tool.poetry.dev-dependencies]

[build-system]
requires = ["poetry-core>=1.0.0"]
build-backend = "poetry.core.masonry.api"

[tool.poetry.scripts]
start = "app.sharehouse.main:start"
```


The full file is exposed in the file `service-ping-metrics-check/pyproject.toml` as a part of Service ping metrics check project.

Idioms

A programming idiom, put simply, is nothing else but a way to write code. Idiomatic Python code is often referred to as being Pythonic. Although there is usually one - and preferably not only one - obvious way to do it. The way to write idiomatic Python code can be non-obvious to Python beginners.

So, good idioms must be consciously acquired.

Explicit code

While (almost) any kind of magic is possible with Python, the most explicit and straightforward manner is preferred. Keep it simple and smart.

python

Bad

```
def foo(args):
    x, y = args
    return dict(locals())
```

Good

```
def bar(x, y):
    return {'x': x, 'y': y}
```

Function arguments

Arguments can be passed to routines in four different ways:

1. Positional arguments - for instance, `foo(message, recipient)`
2. Keyword arguments - for instance, `foo(message, to, cc=None, bcc=None)`. Here `cc` and `bcc` are optional, and evaluate to `None` when they are not passed another value.
3. Arbitrary argument list (`args`)
4. Arbitrary keyword argument dictionary (`kwargs`)

It is up to the engineer writing the function to determine which arguments are positional arguments and which are optional keyword arguments, and to decide whether to use the advanced techniques of arbitrary argument passing. If the advice above is followed wisely, it is possible and enjoyable to write Python functions that are:

- easy to read (the name and arguments need no explanations)
- easy to change (adding a new keyword argument does not break other parts of the code)

Returning values

When a function grows in complexity, it is not uncommon to use multiple return statements inside the function's body.

However, in order to keep a clear intent and sustainable readability, it is preferable to avoid returning meaningful values from many output points in the body.

When a function has multiple main exit points for its normal course, it becomes difficult to debug the returned result, so it may be preferable to keep a single exit point

python

Bad! Probably too complex and difficult to read

```
def foo(a, b, c):
    if not a:
        return None    Raising an exception might be better
    if not b:
        return None    Raising an exception might be better
    Some complex code trying to compute x from a, b and c
    Resist temptation to return x if succeeded
    if not x:
        Some Plan-B computation of x
    return x    One single exit point for the returned value x will help
                when maintaining the code.
```

Good

```
def bar(a, b, c):
    res = None
    if not a:
        res = None
    if not b:
        res = None
    Some complex code trying to compute x from a, b and c
    Resist temptation to return x if succeeded
    if not x:
        Some Plan-B computation of x
        res = 42
    return res
```

Unpacking

If you know the length of a list or tuple, you can assign names to its elements with unpacking. For example, since `enumerate()` will provide a tuple of two elements for each item in list:

python

Bad! - can be difficult to read and maintain

```
for index in range(0:len(foolist)):
    do something with foolist[index]
```

Good! - this is optimized way to use for loop

```
for index, item in enumerate(foolist):
    do something with index and item
```

You can use this to swap variables as well:

```
python
Good
a, b = b, a
```

Nested unpacking works fine as well:

```
python
Good
a, (b, c) = 1, (2, 3)
```

New method of extended unpacking was introduced by PEP 3132:

```
python
Good
a, rest = [1, 2, 3]
a = 1, rest = [2, 3]
a, middle, c = [1, 2, 3, 4]
a = 1, middle = [2, 3], c = 4
```

Ignored variable can be part of unpacking as well:

```
python
Bad
a, , c = [1, 2, 3, 4] This will raise an error
```

```
Good! This will work ( is going before )
a, , c = [1, 2, 3, 4]
```

```
Good
, rest = [1, 2, 3]
rest = [2, 3]
```

> Note: bad praxis is to unpack more than 3 values. Yes, it is allowed to do that, but will rapidly decrease code readability.

```
Python
Bad! (if you have more than 3 values)
a, b, c, d = 1, 2, 3, 4
```

```
Good
a, b, c = 1, 2, 3
d = 4
```

```
Better
a = 1
b = 2
```

```
c = 3
d = 4
```

Conventions

This section should expose effective techniques to deal with conventions and how to integrate them into your toolbox.

Check if a variable equals a constant

It is not needed to explicitly compare a value to True, or None, or 0 · you can just add it to the if statement. See Truth Value Testing for a list of what is considered false.

python

Bad

```
if attr == True:
    print('True!')
```

```
if attr == None:
    print('attr is None!')
```

Good

Just check the value

```
if attr:
    print('attr is truthy!')
```

or check for the opposite

```
if not attr:
    print('attr is falsey!')
```

or, since None is considered false, explicitly check for it

```
if attr is None:
    print('attr is None!')
```

same goes for dict, list sets

```
checklist = []
if checklist:
    print('This is not empty list.')
else:
    print('The list is empty.')
```

String concatenation

There are many ways to do string concatenation. Here is just a short exercise on how to do that in an efficient way.

python

```
string1 = 'Python'
```

```
string2 = 'Guideline'
```

Bad

```
print(string1 + " " + string2)  
Python Guideline
```

Good

```
print('{} {}'.format(string1, string2))  
Python Guideline
```

Better

```
print(f'{string1} {string2}')  
Python Guideline
```

As you noticed here, the result is the same, but details make a difference. In the example above, we dealt with strings, but what will gonna happen when we introduce more data types within the same code.

python

```
string1 = 'Python'  
int1 = 2 now, this is int
```

Bad

```
print(string1 + " " + int1)  
TypeError: can only concatenate str (not "int") to str
```

Good

```
print('{} {}'.format(string1, int1))  
Python 2
```

Better

```
print(f'{string1} {int1}')  
Python 2
```

See why it is better to use placeholders instead of simple string concatenation (ie. a + b).

Short Ways to Manipulate Lists

- List comprehensions provides a powerful, concise way to work with lists.
- Generator expressions follows almost the same syntax as list comprehensions but returns a generator instead of a list. This is crucial to remember: performance and memory matter, and it is a great consideration to understand the leverage of using generators where it is appropriate.

python

Bad

will return a list first and then do the max calculation, the trick is as [] stands for the

list

```
foo = max([(student.id, student.name) for student in graduates])
```

Good

will return a generator object first and then do the max calculation till the generator exhausted, the trick is as () stands for the generator object

```
bar = max((student.id, student.name) for student in graduates)
```

> Note: performance and memory resources matter!

Comprehensions

Comprehensions in Python provide us with a short and concise way to construct new sequences (such as lists, set, dictionary etc.) using sequences which have been already defined. Python supports the following 4 types of comprehensions:

- List Comprehensions
- Dictionary Comprehensions
- Set Comprehensions
- Generator Comprehensions - Kindly suggest spending some time to understand generators and how they are implemented in Python as they are memory optimized and should be an efficient choice for processing a massive set of data.

In other words, any iterable can be part of comprehensions.

Comprehensions are the great example of Pythonic way of thinking, as provides clean and neat coding standards.

python

```
outputlist = [outputexp for var in inputlist if (var satisfies this condition)]
```

> Note: nested comprehensions are allowed as well, but it is not best practice to have more than 2 comprehensions in one statement.

Filtering an iterables

There are plenty of ways to filter an iterables. Let see some of them and how they fit in a high coding standards.

python

Bad

Never remove items from a list while you are iterating through it

Filter elements greater than 4

```
foo = [3, 4, 5]
```

```
for i in foo:
```

```
    if i > 4:
```

```
        foo.remove(i)
```

Bad

Don't make multiple passes through the list

```
while i in foo:  
    foo.remove(i)
```

Good

Use a list comprehension or generator expression

comprehensions create a new list object

```
filteredvalues = [value for value in sequence if value != x]
```

generators don't create another list

```
filteredvalues = (value for value in sequence if value != x)
```

Good

you can use function as a filter

```
sequence= [1, 2, 3]
```

```
def dummyfilter(member: int) -> bool:  
    return member != 2
```

```
filteredvalues = [value for value in sequence if dummyfilter(value)]  
[1, 3]
```

As an alternative, map | filter | reduce functions can be used for this purpose. Handy link is [Map, Filter and Reduce](#)

Modifying the values in a list

Remember that assignment never creates a new object. If two or more variables refer to the same list, changing one of them changes them all.

python

Add three to all list members

```
lista = [3, 4, 5]
```

```
listb = lista  lista and listb refer to the same list object
```

```
for i in range(len(lista)):  
    lista[i] += 3  listb[i] also changes
```

for copying a list, use .copy() method

```
lista = [1, 2, 3]
```

```
listb = lista.copy()
```

extend listb as lista will stay in a original shape

```
listb.extend(listb)
```

```
print(F"lista: {lista}")
```

```
print(F"listb: {listb}")
lista: [1, 2, 3]
listb: [1, 2, 3, 1, 2, 3]
```

It's safer to create a new list object and leave the original alone.

```
python
lista = [3, 4, 5]
listb = lista
```

assign the variable "lista" to a new list without changing "listb"

```
lista = [i + 3 for i in lista]
```

Use enumerate() keep a count of your place in the list.

```
Python
Good
foo = [3, 4, 5]
for i, item in enumerate(foo):
    print(i, item)
prints
0 3
1 4
2 5
```

> Note: The enumerate() function has better readability than handling a counter manually. Moreover, it is better optimized for iterators.

Read From a File

Always good advice is to use context manager over value assigning when loading data from a file. This approach will automatically close a file for you.

```
python
Bad
f = open('file.txt')
a = f.read()
print(a)
f.close() we always forgot something like this.
```

```
Good
with open('file.txt') as f:
    for line in f:
        print(line)
This approach will close a file for you
```


Line Continuations/Line Length

When a logical line of code is longer than the accepted limit, you need to split it over multiple physical lines. The Python interpreter will join consecutive lines if the last character of the line is a backslash. This is helpful in some cases, but should usually be avoided because of its fragility: a white space added to the end of the line, after the backslash, will break the code and may have unexpected results

python

Bad

```
myverybigstring = """When a logical line of code is longer than the accepted limit, \
    you need to split it over multiple physical lines. \
    The Python interpreter will join consecutive lines if the last character of the line is a \
    backslash."""
```

```
from some.deep.module.inside.a.module import anicefunction, anothernicefunction, \
    yetanothernicefunction
```

Good

```
myverybigstring = (
    "When a logical line of code is longer than the accepted limit, "
    "you need to split it over multiple physical lines. "
    "The Python interpreter will join consecutive lines if the last character of the line is a \
    backslash."
)
```

```
from some.deep.module.inside.a.module import (
    anicefunction, anothernicefunction, yetanothernicefunction)
```

Spacing

Following PEP8 we recommend you put blank lines around logical sections of code. When starting a for loop or if/else block, add a new line above the section to give the code some breathing room. Newlines are cheap - brain time is expensive.

python

Bad

```
def foo(inputnumber:int) -> int:
    """
    Do some simple comparing
    """
    res = inputnumber
    if res == 2:
        return res
    else:
        return res 2
```

Good

```
def bar(inputnumber:int) -> int:
```

```
"""
```

Do some simple comparing

```
"""
```

```
res = inputnumber
```

```
if res == 2:
```

```
    return res
```

```
else:
```

```
    return res - 2
```

Type Hints

All function signatures should contain type hints, including for the return type, even if it is None.

This is good documentation and can also be used with mypy for type checking and error checking.

python

Bad

```
def foo(x, y):
```

```
    """
```

Add two numbers together and return.

```
    """
```

```
    return x + y
```

Good

```
def foo(x: int, y: int) -> int:
```

```
    """
```

Add two numbers together and return.

```
    """
```

```
    return x + y
```

Good! (for None as return type)

```
def bar(somestr: str) -> None:
```

```
    """
```

Print a string.

```
    """
```

```
    print(somestr)
```

```
    return
```

Import Order

Imports should follow the PEP8 rules and furthermore should be ordered with any import ... statements coming before from import ...

```
python
Bad
from os import environ
import logging

import somelocalmodule

from requests import get
import pandas as pd
from anotherlocalmodule import something
import sys
```

```
python
Good
import logging
import sys
from os import environ

import pandas as pd
from requests import get

import somelocalmodule
from anotherlocalmodule import something
```

Also, linters should help you with this issue: isort, mypy, flake8, pylint.

isort

isort is a Python utility / library to sort imports alphabetically, and automatically separated into sections and by type. It provides a command line utility, Python library and plugins for various editors to quickly sort all your imports.

Installation:

```
bash
pip install isort
```

Usage:

```
bash
isort filename.py
or isort .
```

and it will automatically order imports with the best practices.

Docstrings

- Docstrings should be used in every single file.
- Docstrings should be used in every single function. Since we are using type hints in the function signature there is no requirement to describe each parameter.
- Docstrings should use triple double-quotes and use complete sentences with punctuation.

python

Good

```
def foo(x: int, y: int) -> int:
```

```
    """
```

```
    Add two numbers together and return the result.
```

```
    """
```

```
    return x + y
```

Good! (for None as return type)

```
def bar(somestr: str) -> None:
```

```
    """
```

```
    Print a string.
```

```
    This is another proper sentence.
```

```
    """
```

```
    print(somestr)
```

```
    return
```

Better! Have Docstring on a module level

```
"""
```

```
This is a Docstrings on a module level.
```

```
Should be handy to describe a purpose of your module
```

```
"""
```

```
def bar(somestr: str) -> None:
```

```
    """
```

```
    Print a string.
```

```
    This is another proper sentence.
```

```
    """
```

```
    print(somestr)
```

```
    return
```

How to integrate Environment Variables

To make functions as reusable as possible, it is highly discouraged (unless there is a "very" good reason) from using environment variables directly in functions (there is an example of this below).

Instead, the best practice is to either pass in the variable you want to use specifically or pass all of the environment variables in as a dictionary.

This allows you to pass in any dictionary and have it be compatible while also not requiring the variables to be defined at the environment level.

```
python
import os
from typing import Dict
```

```
Bad
def foo(x: int) -> int:
    """
    Add two numbers together and return.
    """

    return x + os.environ["y"]
foo(1)
```

```
Good
envvars = os.environ.copy() The copy method returns a normal dict of the env vars.
def bar(somestr: str, anotherstring: str) -> None:
    """
    Print two strings concatenated together.
    """
    print(f'{somestr} {anotherstring}')
    return

bar("foo", envvars["bar"])
```

```
Better
def bar(somestr: str, envvars: Dict[str, str]) -> None:
    """
    Print two strings concatenated together.
    """
    print({somestr} + {envvars["anotherstring"]})
    return

bar("foo", envvars)
```

Parsing Dates

Ideally, never hardcode the date format using `datetime.strptime` unless absolutely necessary in cases where the format is very unusual. A better solution is to use the generic date parser in the `dateutil` library, as it handles a large variety of formats very reliably:

```
python
Bad
datevar = datetime.strptime(tstamp, timestampformat = "%Y-%m-%dT%H:%M:%S%z")

Good
from dateutil import parser as dateparser
...
datevar = dateparser.parse(tstamp)
```

Package Aliases

We use a few standard aliases for common third-party packages. They are as follows:

- import pandas as pd
- import numpy as np

Variable Naming Conventions

Adding the type to the name is good self-documenting code.

When possible, always use descriptive naming for variables, especially with regards to data type. Here are some examples:

- datadf is a dataframe
- paramsdict is a dictionary
- retriesint is an int
- bashcommandstr is a string

If passing a constant through to a function, name each variable that is being passed so that it is clear what each thing is.

Lastly, try and avoid redundant variable naming.

python

```
def bar(somestr: str, anotherstring: str) -> None:
```

```
    """
```

```
    Print two strings concatenated together.
```

```
    """
```

```
    print(somestr + anotherstring)
```

```
    return
```

Good

```
bar(somestr="foo", anotherstring="bar")
```

Better

```
somestr = "foo"
```

```
anotherstring = "bar"
```

```
bar(somestr, anotherstring)
```

But Bad

```
bar(somestr=somestr, anotherstring=anotherstring)
```

Making your script executable

If your script is not able to be run even though you've just made it, it most likely needs to be executable. Run the following:

```
bash
chmod 755 yourscrip.py
```

For an explanation of chmod 755 read this [askubuntu](#) page.

Mutable default function arguments

Using mutable data structures as default arguments in functions can introduce bugs into your code. This is because a new mutable data structure is created once when the function is defined, and the data structure is used in each successive call.

```
python
def appendto(element, to=[]):
    to.append(element)
    return to
```

```
mylist = appendto(12)
print(mylist)
```

```
myotherlist = appendto(42)
print(myotherlist)
```

Output:

```
console
[12]
[12, 42]
```

> Note: Handy link for this topic: [Python gotchas](#)

Exception handling

When writing a python class to extract data from an API it is the responsibility of that class to highlight any errors in the API process. Data modelling, source freshness and formatting issues should be highlighted using dbt tests.

Avoid use of general try/except blocks as it is too broad, and will be difficult to find the real error:

```
python
Bad
try:
    print("Do something")
except:
    print("Caught every type of exception")
```

```
Good
while maximumbackoffsec > (2 - n):
```

```

try:
    print("Do something")
except APIError as gspreaderror:
    if gspreaderror.response.statuscode in (429, 500, 502, 503):
        self.waitexponentialbackoff(n)
        n = n + 1
    else:
        raise
else:
    error(f"Max retries exceeded, giving up on {filename}")

```

Better! fine error granulation

```

while maximumbackoffsec > (2 * n):
    try:
        print("Do something")
    except APIError as gspreaderror:
        if gspreaderror.response.statuscode in (429, 500, 502, 503):
            self.waitexponentialbackoff(n)
            n = n + 1
        else:
            raise
    except AttributeError as attributeerror:
        raise
    except KeyError as keyerror:
        print('Caught this error: ' + repr(keyerror))

```

Folder structure for new extracts

- All client specific logic should be stored in /extract folder, any API Clients which may be reused should be stored in /orchestration folder under the /analytics repo
- Pipeline specific operations should be stored in /extract.
- The folder structure in extract should include a file called extract{source}{datasetname} like extractqualtricsmailingsends or extractqualtrics if the script extracts multiple datasets. This script can be considered the main function of the extract, and is the file which gets run as the starting point of the extract DAG.

Unit Testing with pytest

Pytest is used to run unit tests in the /analytics project. The tests are executed from the root directory of the project with the pythonpytest CI pipeline job. The job produces a JUnit report of test results which is then processed by GitLab and displayed on merge requests.

Most functional tests frameworks, and pytest as well, follow the Arrange-Act-Assert model:

- Arrange - or set up, the conditions for the test
- Act - by calling some function or method
- Assert - that some end condition is True (a test will pass) or False (a test will fail)

pytest simplifies testing workflow by allowing you to use Python's assert keyword directly

without any boilerplate code.

Writing New Tests

New testing file names should follow the pattern test.py so they are found by pytest and easily recognizable in the repository.

New testing files should be placed in a directory named test, usually under the current working folder. The test directory should share the same parent directory as the file that is being tested. For instance, you are working on integration xyz, probably you should have folder structure like following:

```
bash
a typical folder structure for xyz integration
|-- xyz
|   |-- src
|       |-- init.py
|       |-- execute.py
|   |-- test          here you should put your test files
|       |-- testxyz.py
```

A testing file should contain one or more tests.

Test functions should have test naming pattern in their name.

An individual test is created by defining a function that has one or many plain Python assert statements.

- If the asserts are all True, the test passes.
- If any asserts are False, then the test will fail.

> Note: When writing imports, it is important to remember that tests are executed from the root directory.

In the future, additional directories may be added to the PythonPath for ease of testing as need allows.

Basic Pytest Usage

When create a test case, keep it simple and clear. The main point is to have small test cases be able to keep it consistent and easy to maintain..

```
python
example when test passed
import pytest

def testexamplepass():
    assert 1 == 1
test.py::testexamplepass PASSED

example when test failed
import pytest
```

```
def testexamplefailed():
    assert 1 == 2
```

test.py::testexamplefailed FAILED

Using Pytest Fixtures

pytest fixtures are a way of providing data, configs or state setup to tests. Fixtures are functions that can return a wide range of values, especially for repeatable tasks and config items. Each test functions that depend on a fixture must explicitly accept that fixture as an argument along with decorator `@pytest.fixture`.

```
python
import pytest
```

```
@pytest.fixture()
def myfixture():
    define some boring repeatable task needed for test cases
    return "This is my fixture"
```

```
this will pass
def testexample(myfixture):
    assert myfixture == "This is my fixture"
test.py::testexample PASSED
```

```
this will also pass as myfixture is reused
def testexampleadditional(myfixture):
    assert type(myfixture) == str
test.py::testexampleadditional PASSED
```

Parametrized Test Functions

A great way to avoid code duplication is to use Parametrizing tests and for that purpose, the magic happens behind the `@pytest.mark.parametrize` decorator.

The builtin `pytest.mark.parametrize` decorator enables parametrization of arguments for a test function.

This enables us to test different scenarios, all in one function. We make use of the `@pytest.mark.parametrize` decorator, where we are able to specify the names of the arguments that will be passed to the test function, and a list of arguments corresponding to the names.

```
python
import pytest
```

```
here is the magic word to parametrize more than one scenario
import pytest
```

```
@pytest.mark.parametrize("testvalue, expectedvalue", [("1+1", 2), ("2+3", 5), ("69", 54)])
```

```
def testeval(testvalue, expectedvalue):
    assert eval(testvalue) == expectedvalue
test.py::testeval[1+1-2] PASSED [ 33%]
test.py::testeval[2+3-5] PASSED [ 66%]
test.py::testeval[69-54] PASSED [100%]
```

In other words, you can think of this decorator behaving as a zip function and returning a tuple for the 2 list for more scenarios.

> Note: for the decorator `@pytest.mark.parametrize` the first argument to `parametrize()` is a comma-delimited string of parameter names. The second argument is a list of either tuples or single values that represent the parameter value(s).

Categorizing Tests using marks

In any large test suite, some of the tests will inevitably be slow. They might test timeout behavior, for example, or they might exercise a broad area of the code. Whatever the reason, it would be nice to avoid running all the slow tests when you're trying to iterate quickly on a new feature.

pytest enables you to define categories for your tests and provides options for including or excluding categories when you run your suite. You can mark a test with any number of categories.

Marking tests is useful for categorizing tests by subsystem or dependencies. If some of your tests require access to a network, for example, then you could create a `@pytest.mark.networkaccess` mark for them.

- First, need to define markers in `pytest.ini` file:

```
ini
[pytest]
markers =
    networkaccess: requires network access
    localtest: can run locally
```

- Create a test file

```
python
import pytest

@pytest.mark.networkaccess
def testnetwork():
    assert 1 == 2

@pytest.mark.localtest
def testlocal():
    assert 1 == 1
```

- run just networkaccess test(s):

```
bash
will fail, just to recognize what we run
..$ pytest test.py -m networkaccess
...
collected 2 items / 1 deselected / 1 selected

test.py F
```

- run just localtest test(s):

```
bash
this will pass
..$ pytest test.py -m localtest
1 .
...
collected 2 items / 1 deselected / 1 selected

test.py .
```

Duration Report

If you plan to improve the speed of your tests, then it's useful to know which tests might offer the biggest improvements. pytest can automatically record test durations for you and report the top offenders.

Use the `--durations` option to the pytest command to include a duration report in your test results. `--durations` expects an integer value `n` and will report the slowest `n` number of tests.

```
python
import pytest
from time import sleep
```

```
def testslow():
    sleep(1)  make it sleep 1s
    assert 1 == 1
```

```
def testslower():
    sleep(2)  make it sleep 2s
    assert 1 == 1
```

```
def testslowest():
    sleep(3)  make it sleep 3s
    assert 1 == 1
```

- Calling pytest:

```

bash
..$ pytest test.py --durations=1
= test session starts =
...
collected 3 items

test.py ...
[100%]

= slowest 1 durations =
3.00s call    test.py::testslowest
= 3 passed in 6.03s =

```

Using pytest with exceptions

Sometimes, there is a use case you want to force expectations in pytest module. The solution is fairly simple, here is an example:

```

python
def testnamespacefileerror(usageping):
    """
    Test file loading
    """
    with pytest.raises(FileNotFoundError):
        getmetadatafromfile(filename="THISDOESNOTEXISTS.json")

```

in case this file doesn't exist, the function will raise an exception, and test will pass

Also, the handy option is to enrich `pytest.raises` with match statement
with `pytest.raises(ValueError, match="Raising error to.")`:
`runmetricchecks()`

Using pytest with fake RESTful API

Use the `unittest.mock` library when you need to test your code against a RESTful API. `unittest.mock` allows you define API calls and responses, enabling you to test such behaviors within a test environment. Usage is pretty simple:

```

python
from unittest import mock

import pytest
import requests
import responses

define fake response, use fixture mechanism
@pytest.fixture(name="fakeresponse")

```

```

def mockedresponses():
    """
    Mock routine to create fake response
    """
    with responses.RequestsMock() as rsps:
        yield rsps

test fake response
def testconvertresponsetojson(fakeresponse):
    """
    Test function: convertresponsetojson
    """

    expected = {"test1": "pro", "test2": "1"}
    fakeresponse.get(
        "https://somegitlabapiurl/test",
        body='{"test1": "pro", "test2": "1"}',
        status=200,
        contenttype="application/json",
    )

    resp = requests.get("https://somegitlabapiurl/test")

    assert resp == expected

Let's combine unittest.mock and pytest.raises
def testgetresponse(utils):
    """
    Force fake url and raise a Connection Error
    """
    with pytest.raises(ConnectionError):
        = utils.getresponse("https://fakeurl/test")

```

Pytest with simulating environment variables

If you need to add environment variables in the pytest code, you should do it with fixtures.

- Option 1: Using environ

```

python
from os import environ

@pytest.fixture(name="envvar")
def fixturedataclassification():
    """
    Create env variables and initialize
    DataClassification object
    """

```

```

environ["SNOWFLAKEPREPDATABASE"] = "PREP"
environ["SNOWFLAKEPRODDATABASE"] = "PROD"
environ["SNOWFLAKELOADDATABASE"] = "RAW"

```

```

usage
def testinitialization(envvar)
...

```

- Option 2: Using mock.patch

```

python
from unittest.mock import patch

@pytest.fixture(autouse=True, name="setenvvariables")
def mocksettingsenvvars():
    """
    Simulate OS env. variables
    """
    with mock.patch.dict(os.environ, {"STARTTIME": "2023-01-01T00:00:00Z"}):
        yield

```

usage is automatically started, as autouse was set to True

Skip long running test

If you have a scenario where you want to skip a specific test in the CI/CD pipeline (ie. it is too large or taking too long), but want to run it locally on demand, you can use skipif command.

```

python
if you type command:
export RUNALL=YES
test will run, otherwise will skip

@pytest.mark.skipif(
    "RUNALL" not in environ,
    reason="Takes too long if run in the pipeline, want to run locally only",
)
def testlongrunningjob():
    ...

```

Beyond pytest: Useful pytest Plugins

When pytest is not able to answer your needs in more complicated scenarios, handy plugins should be found. By now, didn't find any usage outside of pytest in /analytics repo, and it is good to know there are some useful tools that can help you do your work.

- pytest-randomly - Pytest plugin to randomly order tests and control random.seed.
- pytest-cov - This plugin produces coverage reports. Compared to just using coverage run this plugin does some extras: Subprocess support, Xdist support, Consistent pytest behavior
- Full plugin list - list of pytest plugins.

Tools for supporting coding quality

Python community provides a comprehensive set of tools to ensure code quality and high standards for the codebase.

The idea is to list a couple of tools from our toolbox for the code quality and expose some int